

Carlos Garcia

Reto: day 13

<https://github.com/carlos3474004/30-dia-de-codigo/tree/main/hackerrank/day13>

The screenshot shows a HackerRank challenge completion interface. At the top, a dark blue header contains a badge for '30 Days of Code' with two stars, a message 'You have earned 30.00 points!', and a progress bar at 75% (13/15 challenges). Below this is a green banner with 'Congratulations' and a 'Next Challenge' button. The main content area has a dark background. On the left, three test cases are listed: 'Test case 0', 'Test case 1', and 'Test case 2', all marked as passed with green checkmarks. The 'Test case 0' section is expanded, showing a 'Compiler Message' of 'Success'. Below that, the 'Input (stdin)' is displayed in a code block: 'The Alchemist', 'Paulo Coelho', and '248'. A 'Download' link is next to the input. At the bottom, the 'Expected Output' is shown in a code block: 'Title: The Alchemist', 'Author: Paulo Coelho', and 'Print 248'. A 'Download' link is also next to the expected output.

30 Days of Code **

You have earned 30.00 points!
You are now 2 challenges away from the 3rd star for your 30 days of code badge.

75% 13/15

Congratulations

You solved this challenge. Would you like to challenge your friends? [f](#) [t](#) [in](#)

Next Challenge

Test case 0 ✓

Test case 1 ✓

Test case 2 ✓

Compiler Message

Success

Input (stdin) [Download](#)

```
1 The Alchemist
2 Paulo Coelho
3 248
```

Expected Output [Download](#)

```
1 Title: The Alchemist
2 Author: Paulo Coelho
3 Print 248
```

¡Llegamos al **Día 13 de "30 Days of Code"** en HackerRank! Este reto aborda las **Clases Abstractas (Abstract Classes)**.



¿De qué trata el reto?

Una clase abstracta es una clase "plantilla" que no se puede instanciar directamente y define métodos que las clases hijas **deben** implementar obligatoriamente.

En Python, las clases abstractas se implementan usando el módulo `abc` (*Abstract Base Class*). Ya te dan la clase abstracta `Book` con un método decorado como `@abstractmethod` llamado `display()`.

Tu trabajo es crear la clase hija `MyBook` que:

1. **Herede** de `Book`.
2. Tenga un constructor `__init__(title, author, price)` que envíe el título y el autor a la clase padre usando `super().__init__(title, author)` y guarde el precio (`price`).
3. **Implemente el método** `display()` para imprimir las 3 líneas con el formato requerido:
 - Title: <título>
 - Author: <autor>
 - Price: <precio>



Código de la Solución (Limpio para HackerRank)

Reemplaza **todo** el código en el editor de HackerRank por este bloque único para evitar duplicaciones o errores de indentación:

Python

```
from abc import ABCMeta, abstractmethod

class Book(object, metaclass=ABCMeta):
    def __init__(self, title, author, price):
        self.title = title
        self.author = author
        self.price = price

    @abstractmethod
    def display(self):
        pass

# Write MyBook class
class MyBook(Book):
    def __init__(self, title, author, price):
        super().__init__(title, author, price)

    def display(self):
        print("Title: {}".format(self.title))
        print("Author: {}".format(self.author))
        print("Price: {}".format(self.price))

input()
input()
int(input())
```



Puntos clave

- `class MyBook(Book):` Indica que `MyBook` hereda de la clase abstracta `Book`.
- `super().__init__(title, author):` Reutiliza el constructor de `Book` para asignar el título y el autor.

- `def display(self):` Cumple el "contrato" de la clase abstracta implementando la lógica concreta para imprimir los datos.